# 6.5830/1 Lecture 13: MVCC, Recovery 1

3/31/2026

Tianyu Li

litianyu@mit.edu

Many thanks to Andy Pavlo and CMU DB for sharing their materials

# Administrivia

- Lab 3 Bootcamp this week
  - Look out for announcement on piazza
- Lab 2 Write-up Today
- Quiz 1, Lab 1 grades out

# Last Class

- Optimistic Concurrency Control (OCC) uses timestamps, assigned during the validation phase, to ensure serializability instead of locks.
  - Txns first write changes into private workspaces and then attempt to install them into the database upon commit.

- Phantom Reads occur when a txns re-reads a range of data and finds new rows or missing rows.
  - Achieving Serializability is hard, and systems often settle for weaker isolation levels
  - What is the modern standard instead of serializability?

# Multi-Version Concurrency Control

- The DBMS maintains multiple **physical** versions of a single **logical** object in the database:
  - When a txn writes to an object, the DBMS creates a new version of that object.
  - When a txn reads an object, it reads the newest version that existed when the txn started.
  - Use timestamps to determine visibility.

- **Writers do not block readers.**
  **Readers do not block writers.**

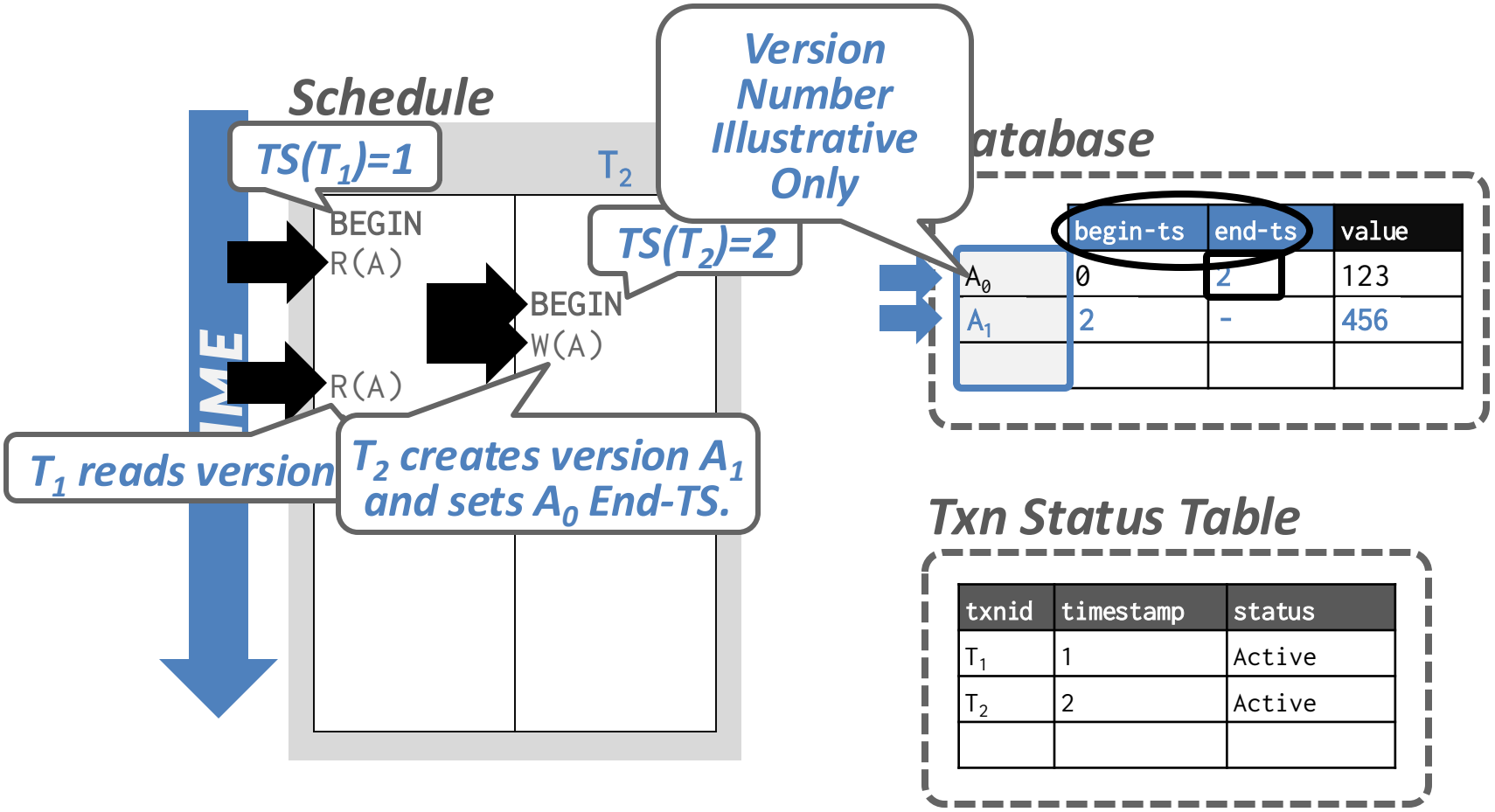
# MVCC History

- Protocol was first proposed in 1978 MIT PhD dissertation.
  - Funny enough, by the designer of UDP
- First implementations was Rdb/VMS and InterBase at DEC in early 1980s.
  - DEC Rdb/VMS is now "Oracle Rdb".
  - InterBase was open-sourced as Firebird.

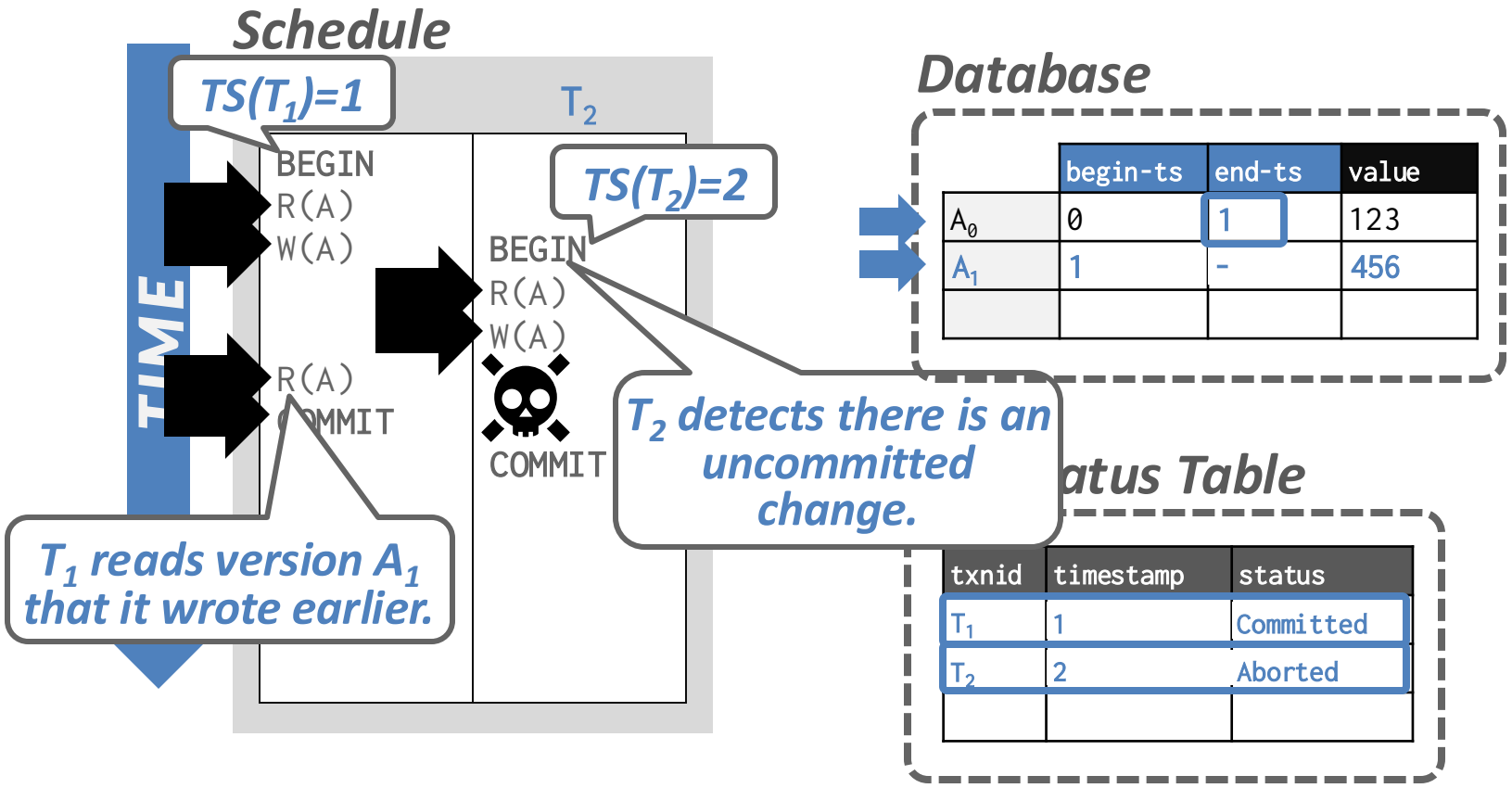
# Snapshot Isolation

- When a txn starts, it sees a consistent snapshot of the database that existed when that the txn started.
  - No torn writes from active txns.
  - If two txns update the same object, then first writer wins.

# MVCC: Example #1



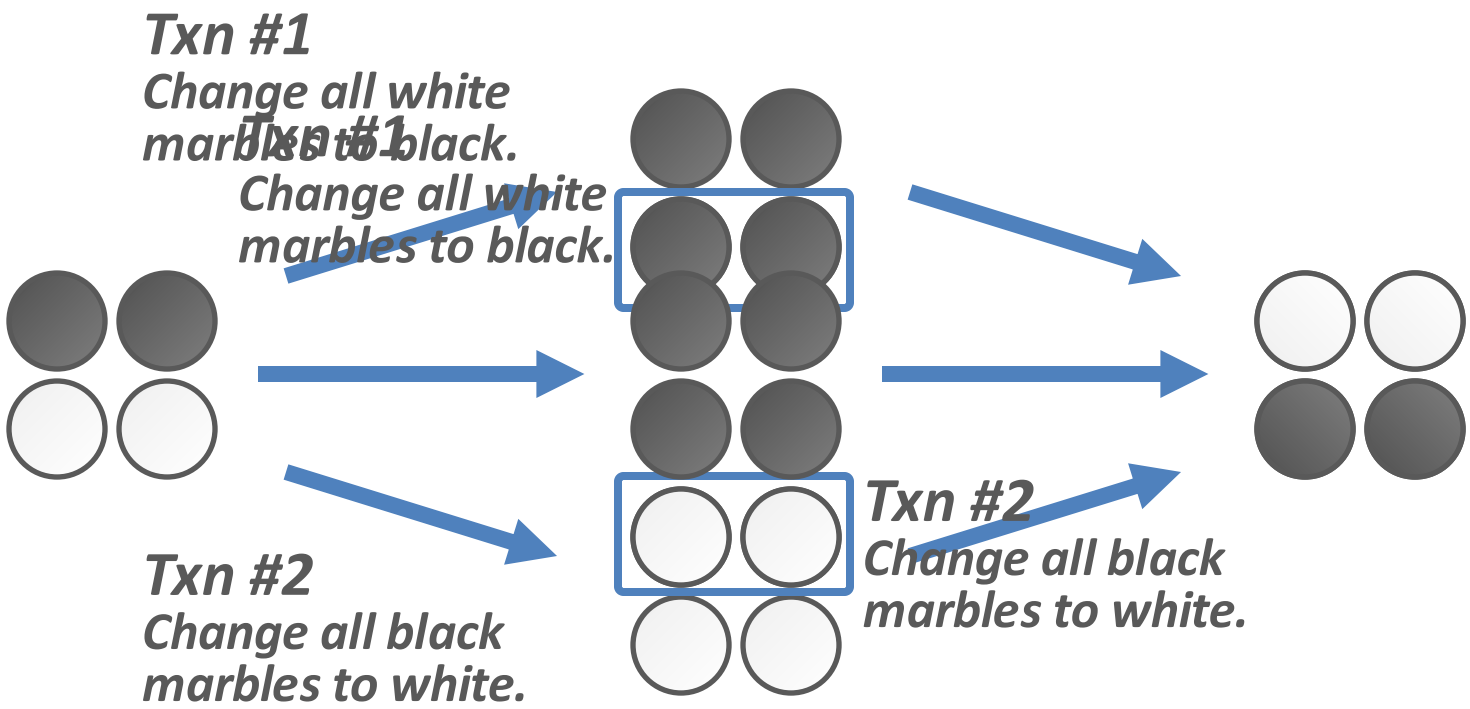
# MVCC: Example #2



# Snapshot Isolation

- When a txn starts, it sees a consistent snapshot of the database that existed when that the txn started.
  - No torn writes from active txns.
  - If two txns update the same object, then first writer wins.
- SI is susceptible to the **Write Skew Anomaly**.

# Write Skew Anomaly



# MVCC -- Implementation

- MVCC is a standalone design decision
  - Can be paired with locking/OCC, precision locks, index locks etc.

- Many implementation choices and intricacies
  - Version storage
  - Garbage Collection
  - Index Management

# MVCC Implementations

| | *Protocol* | *Version Storage* | *Garbage Collection* | *Indexes* |
|---|---|---|---|---|
| Oracle | **MV2PL** | **Delta** | **Vacuum** | **Logical** |
| Postgres | **MV-2PL/MV-TO** | **Append-Only** | **Vacuum** | **Physical** |
| MySQL-InnoDB | **MV-2PL** | **Delta** | **Vacuum** | **Logical** |
| MSSQL Hekaton | **MV-OCC** | **Append-Only** | **Cooperative** | **Physical** |
| SingleStore | **MV-OCC** | **Delta** | **Vacuum** | **Physical** |
| SAP HANA | **MV-2PL** | **Time-travel** | **Hybrid** | **Logical** |
| DuckDB | **MV-OCC** | **Delta** | **Txn-Level** | **Logical** |
| HyPer | **MV-OCC** | **Delta** | **Txn-level** | **Logical** |
| CockroachDB | **MV-2PL** | **Delta (LSM)** | **Compaction** | **Logical** |

# MVCC - Takeaways

- MVCC is (probably) the most widely used scheme in modern DBMSs.

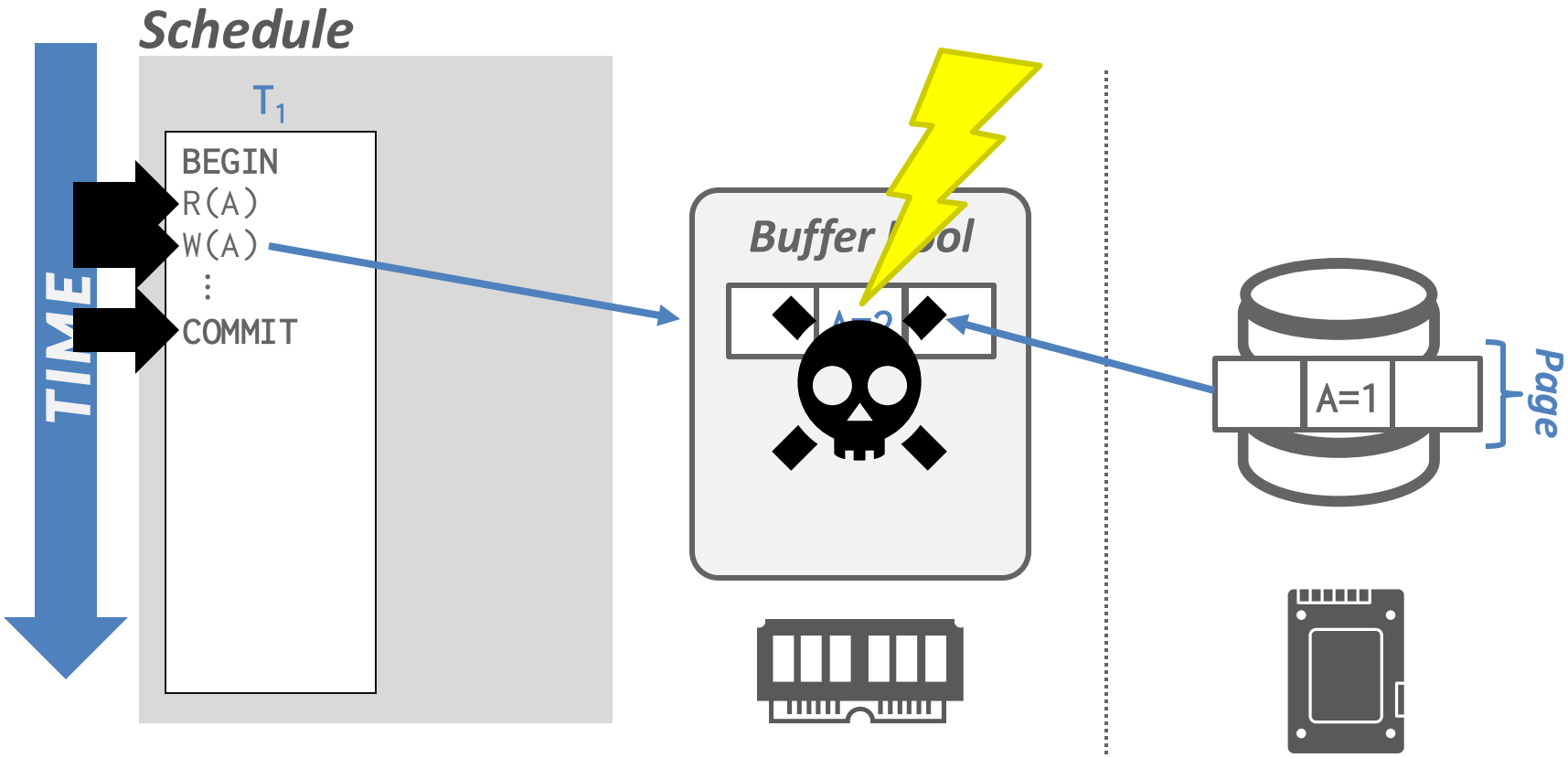
- No time to cover details in this lecture, but MVCC is important to understand when tuning your production databases
  - E.g., Postgres uses append-only version storage

# Next Up: Recovery

- A DBMS's concurrency control protocol gives it **Atomicity + Consistency + Isolation.**

- We now need ensure **Durability**…

# Motivation



# Crash Recovery

- Recovery algorithms are techniques to ensure database consistency, transaction atomicity, and durability despite failures.
- Recovery algorithms have two parts: ***Today***
  - Actions during normal txn processing to ensure that the DBMS can recover from a failure.
  - Actions after a failure to recover the database to a state that ensures atomicity, consistency, and durability.

# Observation

- Durability challenges stem from fast volatile memory & slow durable disk divide
  - On crash, in-memory changes are lost
- The DBMS needs to ensure the following:
  - Writes from a txn are entirely durable once the DBMS reports commit.
  - No partial changes are durable if the txn aborted.

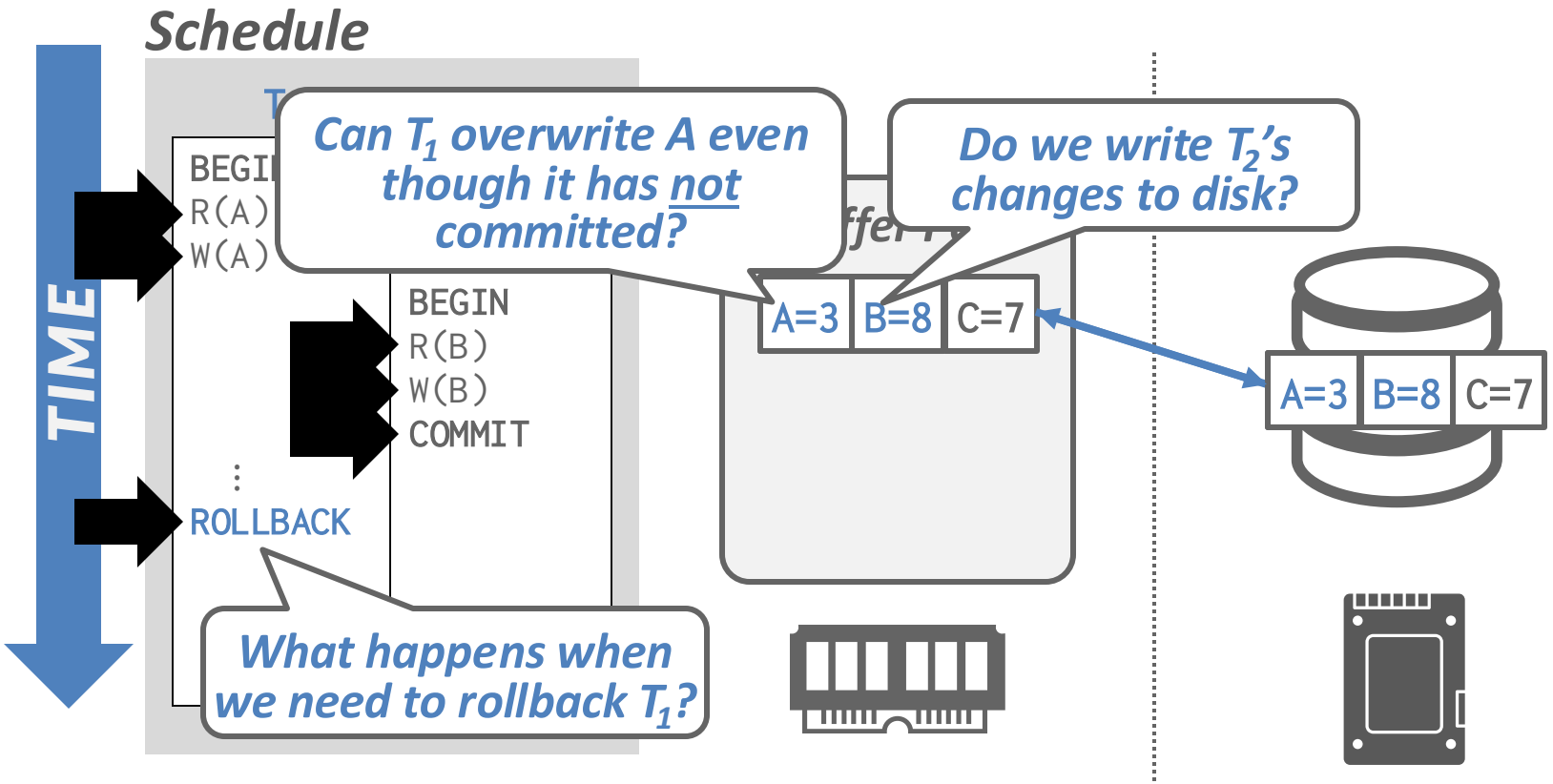
# Undo vs. Redo

- **Undo**: The process of removing the effects of an incomplete or aborted txn.

- **Redo**: The process of re-applying the effects of a committed txn for durability.

- What the DBMS needs to depends on how it manages the buffer pool …

# Example


Schedule
TIME
BEGI
R(A)
W(A)
BEGIN
R(B)
W(B)
COMMIT
ROLLBACK
Can $T_1$ overwrite A even though it has not committed?
Do we write $T_2$'s changes to disk?
What happens when we need to rollback $T_1$?
A=3
B=8
C=7
A=3
B=8
C=7

# STEAL

- Whether the DBMS can evict a dirty object in the buffer pool modified by an uncommitted txn and overwrite the most recent committed version of that object in non-volatile storage.

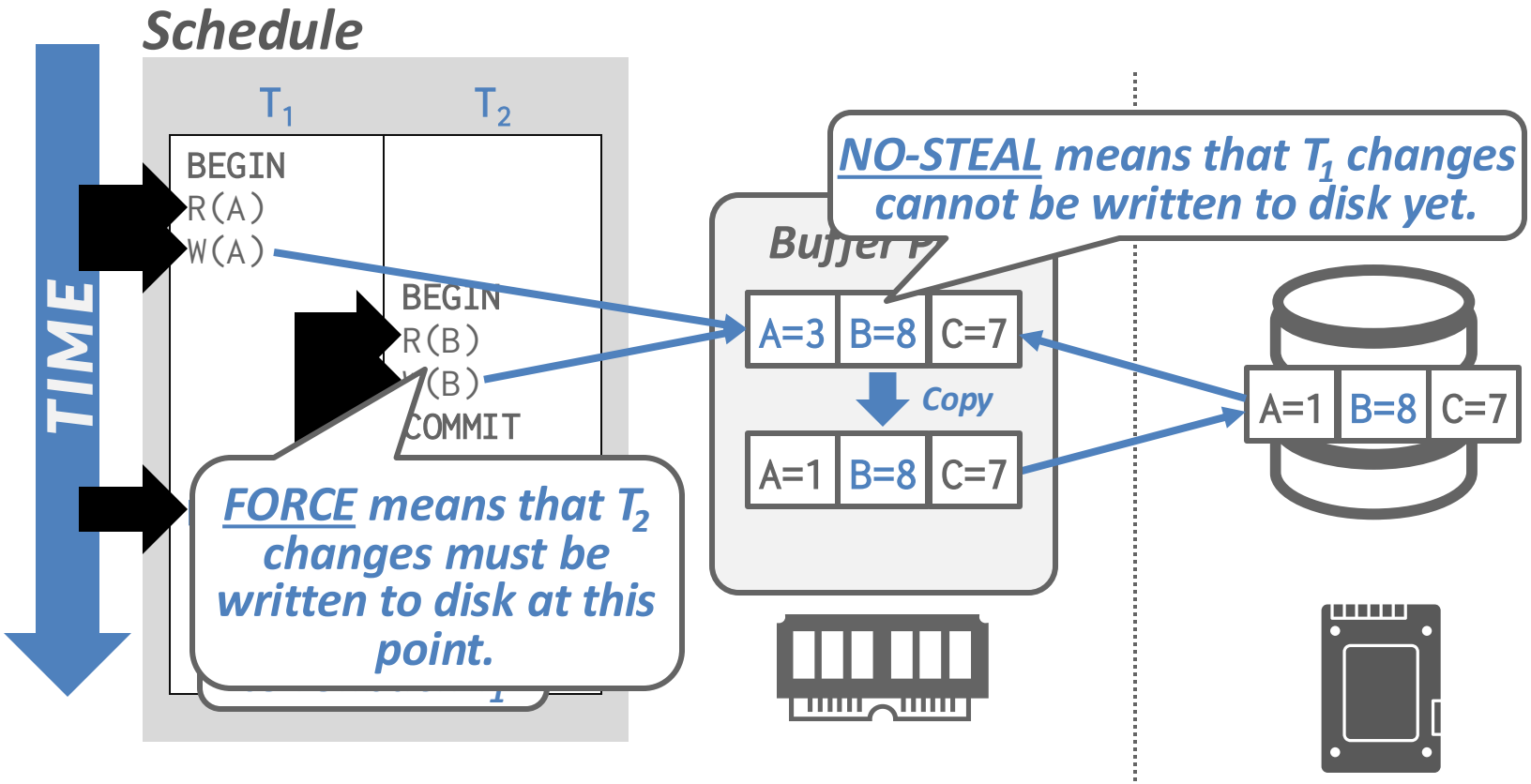
- **STEAL**: Eviction + overwriting is allowed.
- **NO-STEAL**: Eviction + overwriting is not allowed.

# FORCE

- Whether the DBMS requires that all updates made by a txn are written back to non-volatile storage before the txn can commit.
- **FORCE**: Write-back is required.
- **NO-FORCE**: Write-back is not required.

refers to what happens at the moment of commit

# NO-STEAL + FORCE



# NO-STEAL + FORCE

- Trade-offs?
  - Easy to implement: no redo/undo
  - Large overhead of copying on critical path
  - Write Amplification
  - Many random writes
  - Cannot support large write sets
- Also, page writes are not necessarily atomic – need to add additional overhead via double writing, etc.

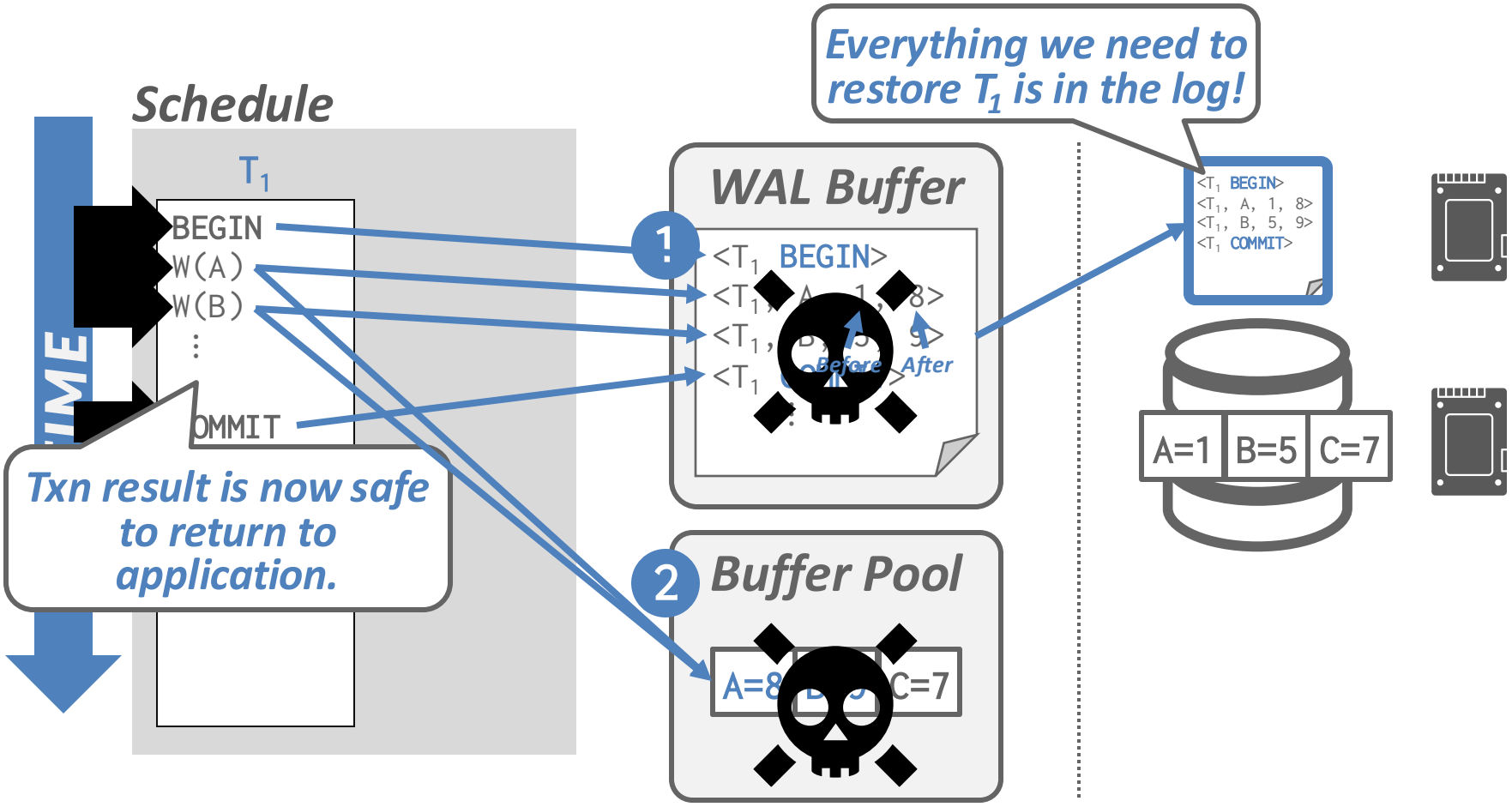
# Idea: Write-Ahead Logging

- Maintain a log file separate from data files that contains the changes that txns make to database.
  - Writes are sequential and incremental
  - Log contains enough information to perform the necessary undo and redo actions to restore the database.
- Buffer Pool Policy: **STEAL** + **NO-FORCE**
  - DBMS must write to disk the log file records that correspond to changes made to a database object before it can flush that object to disk.

# WAL Protocol

- Write a <BEGIN> record to the log for each txn to mark its starting point.
- Append a record every time a txn changes an object:
  - Transaction Id
  - Object Id
  - Before Value (**UNDO**)
  - After Value (**REDO**)
- When a txn finishes, the DBMS appends a <COMMIT> record to the log.
  - Make sure that all log records are flushed before it returns an acknowledgement to application.

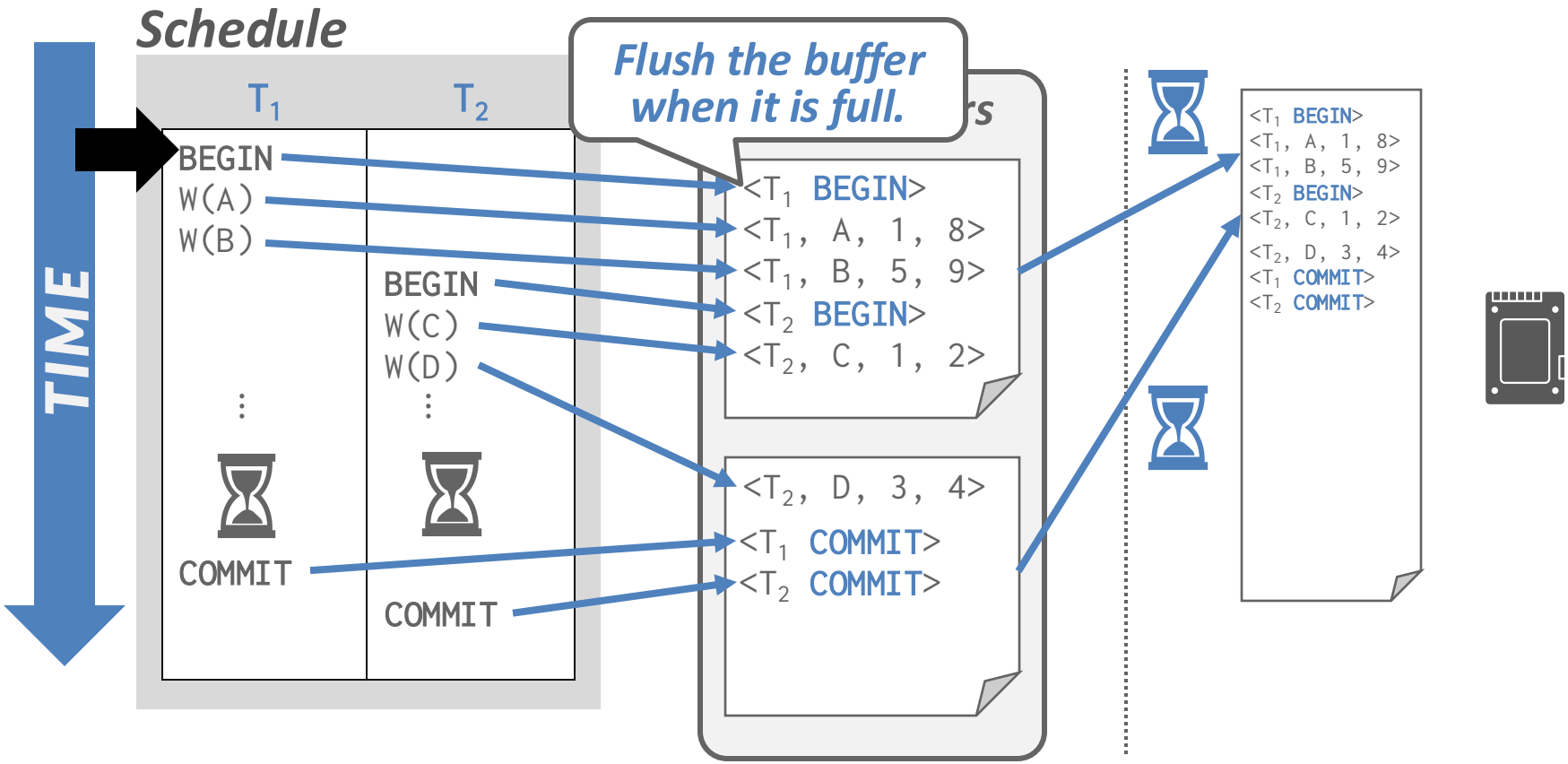
# Example


Schedule
T1
BEGIN
W(A)
W(B)
COMMIT
Txn result is now safe to return to application.
Everything we need to restore T1 is in the log!
WAL Buffer
1
<T1 BEGIN>
Before
After
Buffer Pool
2
A=8
C=7
<T1 BEGIN>
<T1, A, 1, 8>
<T1, B, 5, 9>
<T1 COMMIT>
A=1
B=5
C=7

# Group Commit

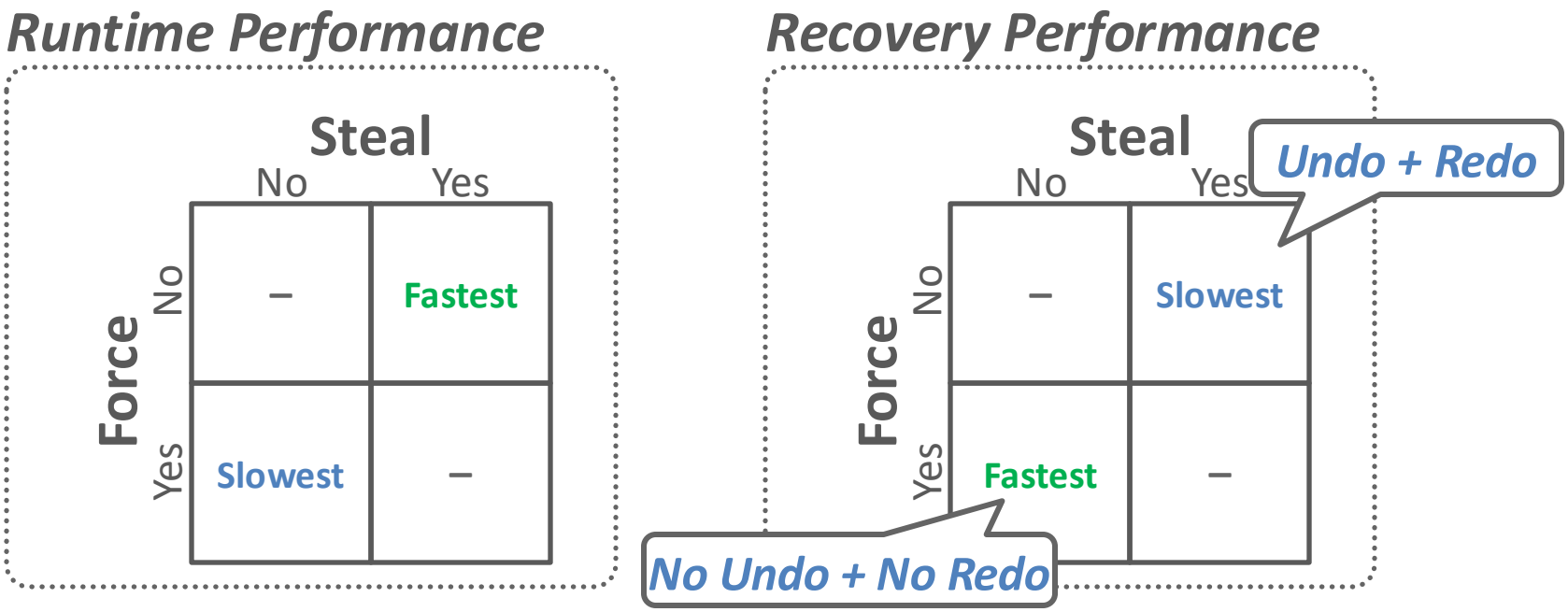
- Flushing the log buffer to disk every time a txn commits will become a bottleneck.
- The DBMS can use the **group commit** optimization to batch multiple log flushes together to amortize overhead.
  - When the buffer is full, flush it to disk.
  - Or if there is a timeout (e.g., 5 ms).

# Group Commit



# Buffer Pool Policies

- Almost every serious DBMS uses **NO-FORCE** + **STEAL**



# Logging Scheme

- **Physical Logging**
  - Record the byte-level changes made to a specific page.
- **Logical Logging**
  - Record the high-level operations executed by txns.
  - E.g.: `UPDATE`, `DELETE`, and `INSERT` queries.

# Trade-offs

- Logical logging requires less data written in each log record than physical logging.
- Difficult to implement recovery with logical logging because logical changes may not be idempotent
  - Re-applying may yield different end state

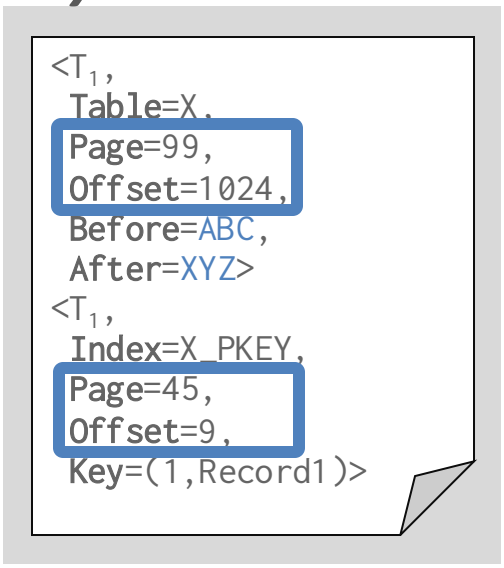
# Physiological Logging

- **Most serious systems use physio-logical logging**
  - Physical-to-a-page, logical-within-a-page.
  - Hybrid approach with byte-level changes for a single tuple identified by page id + slot number.
  - Does not specify organization of the page.
- **Also helps greatly with recovering complex data structures**
  - E.g., an on-disk B-tree that crashed during rebalancing

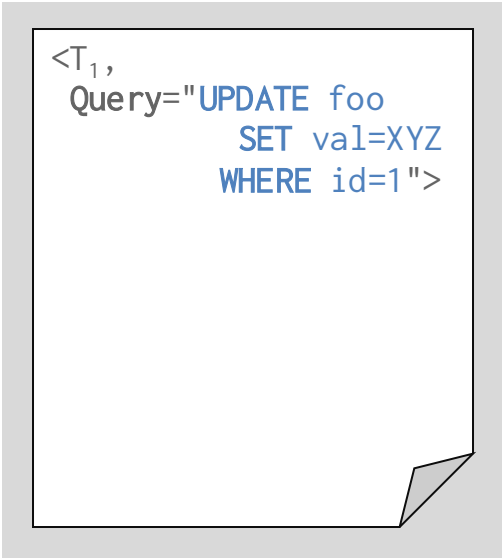
# Logging Schemes

```
UPDATE foo SET val = XYZ WHERE id = 1;
```

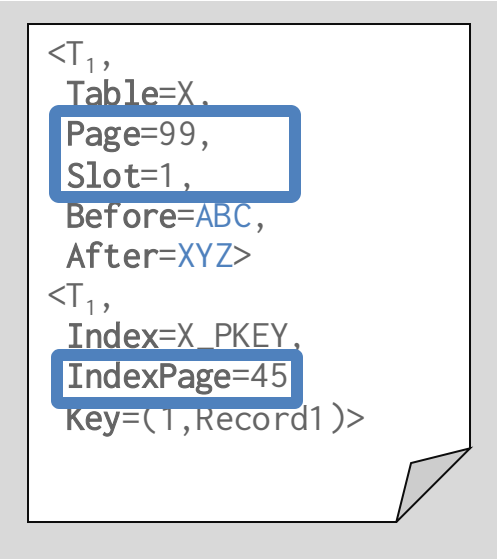
***Physical***




***Logical***

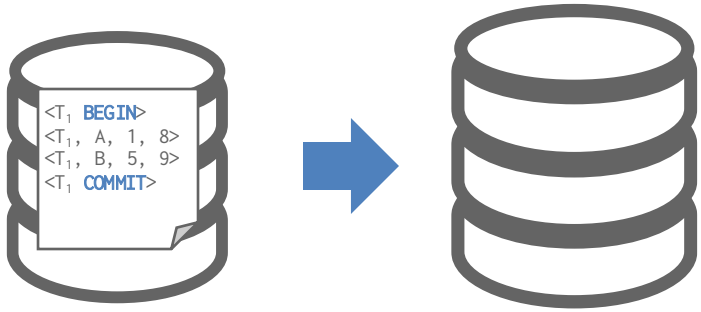



***Physiological***



# CHANGE DATA CAPTURE (CDC)

- Logs are also used to automatically propagate changes to external sources.
  - **Extract Transform Load** (ETL) to an analytical system
  - Synchronization with a replica
  - Cloud-native storage integration (lecture 20)
  - Some systems can do this automatically. Others require third-party tools.



# Observation

- The DBMS's WAL will grow forever.
  - After a crash, the DBMS must replay the entire log, which will take a long time.

- The DBMS periodically takes a **checkpoint** to attempt log truncation
  - Intuitively, if all changes from a prefix of the log are on disk, no need to replay that prefix
  - Need to flush buffers, but also do log bookkeeping

# Strawman

- **Blocking / Consistent Checkpoint Protocol:**
  - Pause all queries.
  - Flush all WAL records in memory to disk.
  - Flush all modified pages in the buffer pool to disk.
  - Write a `<CHECKPOINT>` entry to WAL and flush to disk.
  - Resume queries.

# Strawman: Problems

- DBMS must stall txns when it takes a checkpoint to ensure a consistent snapshot.
- Still need to scan the log to find uncommitted txns at time of checkpoint.
  - Cannot simply drop the entire log up until checkpoint
- Takeaway: Implementing logging + recovery is very tricky and hard to make performant
  - We will discuss exactly how to do it next lecture

# CONCLUSION

- Write-Ahead Logging is (almost) always the best approach to handle loss of volatile storage.
  - Use incremental updates (**STEAL** + **NO-FORCE**) with checkpoints.
  - On Recovery: undo uncommitted txns + redo committed txns.

- We will discuss ARIES next class